3D Spatial Pose Tracking AprilTags

1. Course Overview

The function implemented in this lesson can be simply understood as: **hold the AprilTag and move it anywhere in space, the robotic arm end-effector follows omnidirectionally in 3D like a magnet**.

The system uses visual computation to obtain the AprilTag's 3D spatial coordinates in the camera frame in real-time. Through **three-channel independent PID** (X-axis yaw, Y-axis pitch, Z-axis depth distance), it closed-loop controls the robotic arm's base rotation, up/down lifting, and front/rear pushing/pulling, achieving omnidirectional dynamic following of the gripper end-effector in 3D space.

> **Note**
>
> **This lesson only demonstrates spatial pose tracking, without chassis movement and grasping. Press spacebar in the image window to start or stop tracking mode.**

---

2. Preparation and Running

This case requires opening 2 terminals to start the system sequentially:

2.1 Start Camera Driver and Kinematics Solver Environment

Run in **Terminal 1**:

```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```

```
jetson@yahboom:~$ ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-09-02-16-32-09-195196-yahboom-1117638
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [camera_launch-1]: process started with pid [1117670]
[INFO] [kinemarics_dofbot-2]: process started with pid [1117672]
[INFO] [Ackman_driver_A1P-3]: process started with pid [1117674]
[kinemarics_dofbot-2] ----------------
[kinemarics_dofbot-2] fk
[kinemarics_dofbot-2] 5
[kinemarics_dofbot-2] ----------------
[camera_launch-1] [INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-09-02-16-32-10-139043-yahboom-1117670
[camera_launch-1] [INFO] [launch]: Default logging verbosity is set to INFO
[camera_launch-1] [INFO] [usb_cam_node_exe-1]: process started with pid [1117735]
[camera_launch-1] [usb_cam_node_exe-1] libEGL warning: DRI2: failed to authenticate
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337930.852521184] [usb_cam]: camera_name value: test_camera
[camera_launch-1] [usb_cam_node_exe-1] [WARN] [1788337930.852749618] [usb_cam]: framerate: 120.000000
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337930.876670665] [usb_cam]: camera calibration URL: package://usb_cam/config/cam
era_info.yaml
[camera_launch-1] [usb_cam_node_exe-1] [WARN] [1788337930.879330849] [usb_cam]: [test_camera] does not match narrow_stereo in file /
home/jetson/yahboomcar_ros2_ws/software/library_ws/install/usb_cam/share/usb_cam/config/camera_info.yaml
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337930.953127689] [usb_cam]: Starting 'test_camera' (/dev/video0) at 640x480 via
mmap (mjpeg2rgb) at 120 FPS
[camera_launch-1] [usb_cam_node_exe-1] [swscaler @ 0xaaaad038ffe0] No accelerated colorspace conversion found from yuv422p to rgb24.
[camera_launch-1] [usb_cam_node_exe-1] This device supports the following formats:
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1280 x 720 (60 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1920 x 1080 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1024 x 768 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 640 x 480 (120 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 800 x 600 (60 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1280 x 1024 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 320 x 240 (120 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1280 x 720 (9 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1920 x 1080 (6 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1024 x 768 (6 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 640 x 480 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 800 x 600 (20 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1280 x 1024 (6 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 320 x 240 (30 Hz)
[Ackman_driver_A1P-3] [INFO] [1788337931.035307899] [driver_node]: A1 driver initialized: imu_link=imu_link, pub_hz=20.0, arm_status
_hz=5.0, cmd_vel_timeout=0.50s, pub_odom=True
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337931.748886319] [usb_cam]: Setting 'white_balance_temperature_auto' to 1
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337931.749045211] [usb_cam]: Setting 'exposure_auto' to 3
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337931.760676716] [usb_cam]: Setting 'focus_auto' to 0
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337931.836160385] [usb_cam]: Timer triggering every 8 ms
```

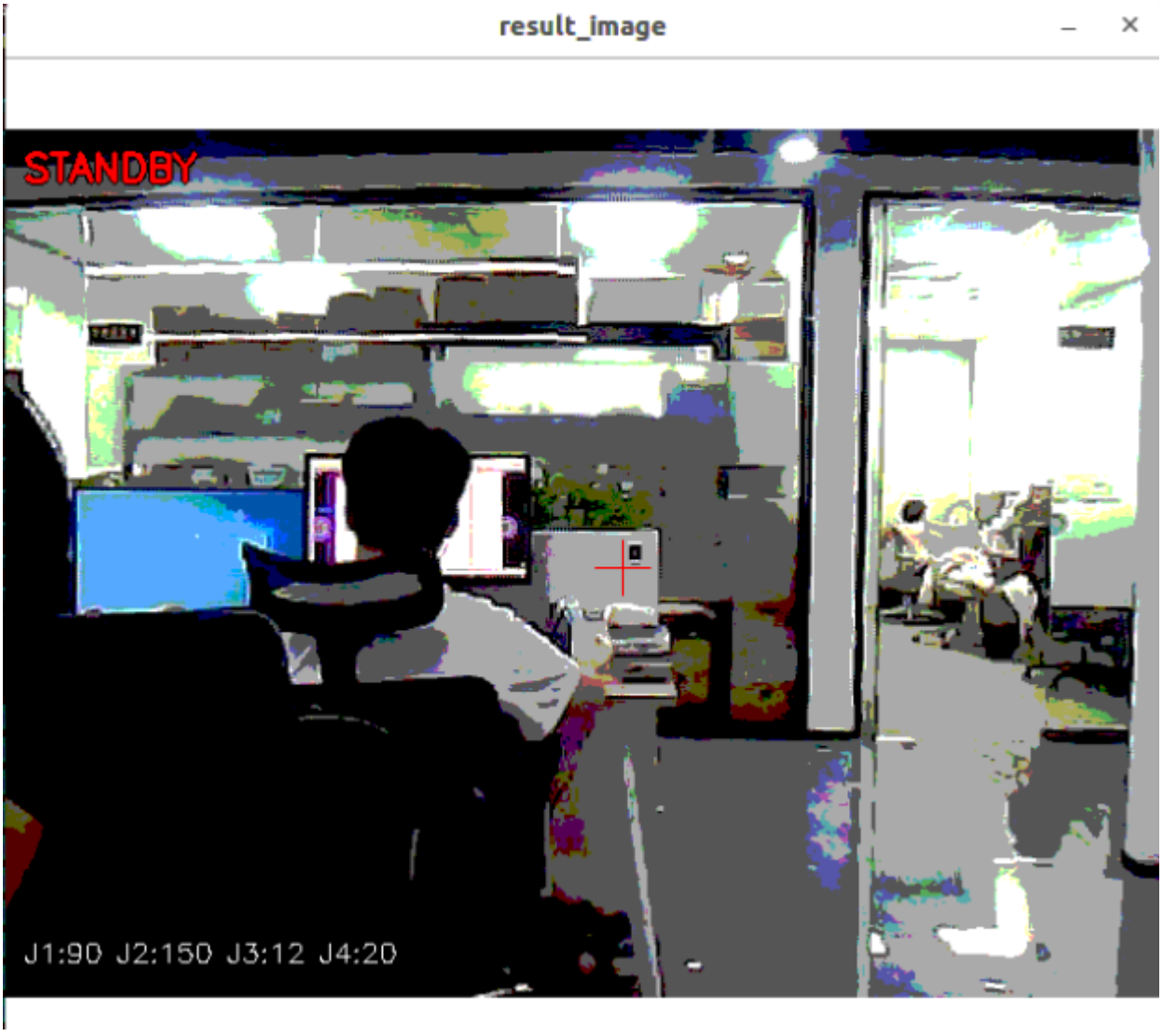
2.2 Start 3D Spatial Tracking Node

Run in **Terminal 2**:

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver apriltag_follow

# Or USB camera version
ros2 run dofbot_pro_usb apriltag_follow
```

```
jetson@yahboom:~$ ros2 run dofbot_pro_usb apriltag_follow
Loaded Offset -> x: 0.009368814795573138, y: 0.007718452024610358, z: 0.0
[INFO] [1788344347.388607368] [ApriltagTrack_node]: tag_size: 0.04 m
[INFO] [1788344347.426186614] [ApriltagTrack_node]: AprilTag Follow (3-Channel PID + USB Camera) In
itialized.
```



3. Operation Demo and Interaction Flow

1. **Initial standby state**: After node starts, the image upper-left corner shows `STANDBY`, robotic arm is stationary at patrol pose;
2. **Start 3D following**:

- Keep the image window active, press the **spacebar (`Space`)**;
- Status switches to highlighted green `TRACKING`, robotic arm starts 3D spatial closed-loop tracking;

3. **Spatial block movement guidance**:
   - Hold the AprilTag block and move it slowly and smoothly in 3D space in front of the robotic arm:
     - **Left/right translation**: Joint 1 base servo rotates left/right smoothly for following;
     - **Up/down lifting**: Joints 2/3/4 pitching coordinate to lift or press down;
     - **Front/rear pushing/pulling**: Robotic arm automatically adjusts link extension to maintain constant lens-to-block distance;
4. **Pause and reset**: Press **spacebar** again, tracking pauses immediately, robotic arm automatically returns to initial pose and resets each channel PID integral.

> **Tip**
>
> Please move **slowly and smoothly**. The USB monocular camera's effective following distance is approximately 0.16m. The 3D depth camera's effective following depth distance is set to 0.35m. Too close or too far can easily exceed the robotic arm's IK limits.

---

4. Core Source Code Analysis

The 3D servo tracking system is driven by underlying vision recognition and three-channel PID algorithm:

Hardware Solution	Spatial Tracking Core Node	Depth Measurement Method
USB Monocular Camera	`dofbot_pro_usb/apriltag_follow.py`	Pinhole perspective model calculates from tag size
3D Depth Camera	`dofbot_pro_driver/apriltag_follow.py`	Hardware-level depth point cloud matrix direct reading

The main technical implementation comparison:

Comparison Dimension	USB Monocular Camera Solution	3D Depth Camera Solution (Nuwa-HP60C)
Target Image Reference	`TARGET_X = 341, TARGET_Y = 242`	`TARGET_X = 320, TARGET_Y = 240`
Reference Spatial Depth	`TARGET_DEPTH = 0.16m`	`TARGET_DEPTH = 0.35m` (wider measurement range)
Depth Channel PID	`pid_z = (2.0, 0.0, 0.5)`	`pid_z = (3.0, 0.0, 0.5)` (more responsive)
Slope Limiter Smoothing	X/Y axis limit 0.015, Z axis limit 0.02	X/Y axis limit 0.015, Z axis limit 0.02

4.1 Three-Channel PID Independent Decoupling Control

Compared with 2D plane tracking, 3D tracking adds independent closed-loop control of the **Z-axis depth channel**, aligning the target center across three dimensions separately:

```
# apriltag_follow.py —— Three-axis PID initialization
self.pid_x_params = (0.35, 0.0, 1.0)  # X axis (left/right rotation adjustment)
self.pid_y_params = (1.00, 0.0, 1.0)  # Y axis (pitch up/down adjustment)
self.pid_z_params = (3.00, 0.0, 0.5)  # Z axis (front/rear push/pull adjustment)

# Error input and calculation
raw_x = self.pid_x.compute(center_x, TARGET_X)
raw_y = self.pid_y.compute(center_y, TARGET_Y)
raw_z = self.pid_z.compute(depth_val, TARGET_DEPTH)
```

4.2 Multi-Joint Feed-Forward Fusion Mapping Algorithm

To prevent the robotic arm's lens pitch perspective from changing during front/rear extension, the depth error calculated `angular_z` is **simultaneously superimposed onto Joint2, Joint3, and Joint4**, achieving a "front/rear push/pull" motion similar to a parallelogram mechanism:

```
# apriltag_follow.py —— XYZ_track() joint fusion
# X yaw: controls joint 1 base left/right rotation
target_j1 = base_j1 + (10 * angular_x * self.gain_j1)

# Y pitch + Z extension: multi-axis coordinated linear superimposition
target_j2 = base_j2 + (angular_y * self.gain_pitch_j2) + (angular_z * self.gain_reach_j2)
target_j3 = base_j3 + (angular_y * self.gain_pitch_j3) + (angular_z * self.gain_reach_j3)
target_j4 = base_j4 + (angular_y * self.gain_pitch_j4) + (angular_z * self.gain_reach_j4)
```

4.3 Independent Slope Limiting and Safe Reset

Each axis is equipped with an independent per-frame maximum change slope limiter (`max_step_change_xy = 0.015`, `max_step_change_z = 0.02`), effectively filtering abrupt impacts from image detection frame drops or hand jitter; when the user presses spacebar again to exit, the system automatically clears integral cache and smoothly returns to position:

```
# apriltag_follow.py —— Spacebar safe switching and reset
if key == 32:
    self.tracking_active = not self.tracking_active
    if not self.tracking_active:
        self.pubSixArm(self.init_joints)  # Reset to safe pose
        self.pid_x.reset()
        self.pid_y.reset()
        self.pid_z.reset()
```

Through this 3D decoupling control architecture, whether using USB monocular lens or 3D depth camera, the robotic arm can present magnet-like tightly smooth dynamic following in 3D space.